背景

神经网络(nn网络)已广泛应用于移动和嵌入式设备。但预训练的模型不满足一些特殊领域有在线训练的需求，比如无人机搜救、个性化面部识别和嵌入式ai驱动机器人等许多应用中，必须在短时间内更新设备上的神经网络模型，以便及时适应新的数据或来自环境的实时反馈。

由于神经网络的计算密集型特性以及大多数移动和嵌入式设备的计算能力有限，神经网络的训练和推理通常需要在云端或者专用的服务器上进行。但这种方法会产生很高的通信开销。

然而，又由于设备的计算能力有限，神经网络模型的设备上训练通常是耗时的。由此可见在移动和嵌入式设备上加速模型的训练速度是十分必要的

> 例如，使用Raspberry Pi 4使用包含约6000张图像的CUB-200数据集训练ResNet50模型可能需要>30天。即使在配备了Nvidia Jetson TX2等gpu的更强大的设备上，对每张图像的训练仍然可能需要30秒。

为了减少设备上训练的长时间延迟，可以选择使用**较小的神经网络模型**，但由于其有限的学习能力，当应用于新的在线数据时，模型性能可能会受到影响。

一种直观的解决方案是利用**硬件加速器**，如专用DSP或NPU，但它们在移动和嵌入式设备上的可用性有限。

传统的迁移学习(1)建议只训练神经网络模型的一部分。它假设大多数顶层已经学习了特征提取的通用能力，并将可训练的部分限制在少数底层。然而，这种限制削弱了神经网络的学习能力，并使其在困难学习任务上的推理准确率降低了20%。

后来的研究通过识别重要的神经网络子结构(例如，偏置参数，归一化层和并行分支)消除了这一限制，但只在离线情况下评估这种重要性，而没有考虑导致动态训练反馈的在线数据的可变性。因此，神经网络子结构的在线重要性可能与离线重要性有很大不同，从而导致明显的精度损失。

现有比较好的方法是在运行时自适应调整可训练神经网络部分。用于设备上训练的神经网络剪枝可以动态地去除不太重要的神经网络结构(2)。然而，被修剪后的神经网络部分即使有用也不能再被选择，神经网络的表示能力就会随着时间的推移而减弱，对于困难的学习任务变得不足。训练也可以从一个逐渐增长的小型神经网络开始(3)，但新增加的神经网络层是从头开始训练的，需要超过30%的额外训练周期。

现有的方法有以下几个问题：

1. 训练的效率
2. 对可训练神经网络部分的选择是单向的和不可恢复的过程。因此，可训练神经网络部分的演化受到约束，不能灵活地适应当前的训练需求。

本论文提出对于神经网络的加速应该是完全弹性的，即每个神经网络子结构都可以根据需要在训练的任何时候自由地从可训练的神经网络部分中移除或添加。

技术要点

为了解决这个挑战，论文提出了一种新的时间模型。
ElasticTrainer使用了Tensor Importance Evaluator模块来评估神经网络张量的重要性，并使用Tensor Timing Profiler模块来精确测量张量的训练时间。然后，基于这个模型，论文开发了一种动态规划算法，可以从指数级的可能性中找到最优的张量选择，计算开销很小。

作者通过考虑不同神经网络操作的硬件加速，实现了ElasticTrainer的训练加速和FLOPs节省。与现有工作中广泛使用的FLOPs不同，ElasticTrainer通过考虑不同神经网络操作的硬件加速来实现实际的训练时间加速。

粒度选择

层选择是粗粒度的，因此不准确，因为层内的一些重要权重可能由于许多其他不重要的权重而被忽视。另一方面，虽然权重级选择是最细粒度的，但它需要细粒度索引和不规则的数据访问模式，这可能导致基于向量和矩阵计算实现的TensorFlow和PyTorch框架的训练效率较差因此，ElasticTrainer采用张量级选择，既保证了准确性，又可以在现有的NN框架中有效地执行，而不需要额外的开销

张量时间分析器

Tensor Timing Profiler（张量计时分析器）是用于分析神经网络张量训练时间的模块。传统的神经网络分析工具可以测量离线训练中每个操作（如矩阵乘法和卷积）的执行时间，但无法准确反映参与这些操作的张量的训练时间。为了解决这个问题，ElasticTrainer首先将原始的基于层的神经网络结构转换为基于张量的计算图，保留了所有张量在训练中的执行顺序。然后，通过聚合相关神经网络操作的时序信息，计算每个张量的反向传播时间tdy和权重更新时间tdw。该模块的关键挑战在于确定张量的执行顺序以及如何聚合相关神经网络操作的时序信息，计算每个张量的反向传播时间tdy和tdw。这取决于张量所在的神经网络层的类型。作者考虑了不同类型的NN层(例如，卷积层、全连接层和归一化层)，并制定了描述每种层类型的规则。

张量重要性评估器

Tensor Importance Evaluator（张量重要性评估器）是用于评估神经网络张量重要性的模块。传统的重要性评估方法通常基于张量的大小，但无法准确反映不同权重之间的依赖关系。为了解决这个问题，ElasticTrainer借鉴了可解释的人工智能（XAI）方法，并扩展了这些方法以评估张量的重要性。该模块**通过计算权重更新的梯度变化**来评估张量的重要性。

> 公式定义了在特定训练时期中张量 k 的重要性评估 I_k ，其中 L 表示训练损失函数， w_i^k 表示张量k中的第i个权重， Δw_i^k 表示训练时期中 w_i^k 的最近更新。该评估指标考虑了每个权重更新对训练损失的贡献，并通过梯度计算自然地融入了权重之间的依赖关系。

> 在公式 (5) 中， `c` 是一个乘法因子，表示对所有 `M` 个神经网络权重的连续撤销操作。计算了关于 `c` 的损失梯度，这涉及到计算损失函数 `L` 关于 `c` 的导数。这是通过应用微分的链式法则完成的，其中 `u` 是更新后的权重，而 `Δw` 表示权重更新。

> 当 `c` 设为0时，公式（5）变为一个重要性向量，其中每个元素对应一个神经网络权重的重要性。这种计算的重要性隐含地包含了权重依赖性的影响。这种方法允许在训练过程中有效地计算权重的重要性。

evaluation interval指的是在训练过程中评估张量重要性的时间间隔。

实验结果

论文在多个嵌入式设备上实现了ElasticTrainer，并在各种常见数据集上评估了其性能。

加速与准确性

ElasticTrainer在实际时间上实现了多达3.5倍的训练速度提升，并减少了60%的训练FLOPs。此外，ElasticTrainer在大多数数据集上几乎没有准确性损失，并且相比现有方案，至少提高了10%的准确性。

实验结果显示，ElasticTrainer在测试准确性方面与Full Training相当，并且在一些数据集上甚至能够获得更高的准确性。相比之下，Traditional TL在CUB-200数据集上的准确性损失超过20%，而BN+Bias在更难的数据集上（如CUB-200）的准确性下降超过10%。PruneTrain虽然可以节省30%以上的训练FLOPs，但由于在运行时需要计算额外的损失函数，其实际的墙钟时间节省并不明显。

此外，ElasticTrainer在训练速度方面也表现出色。在Jetson TX2上，相对于Full Training，ElasticTrainer可以实现2倍至3.4倍的训练速度提升。与BN+Bias相比，ElasticTrainer的训练速度提升高达30%。在Raspberry Pi 4B上，ElasticTrainer相对于Full Training的训练速度提升更为显著。

弹性张量选择

由于传递误差梯度的高时间成本，top tensor (lower indices) 被选择的可能性较小。如果top tensor确实非常重要并且应该被选择，ElasticTrainer可以自适应地跳过更新几个重要性较低的底张量(高指标)，以保持所需的训练加速。

此外，张量选择在更困难的数据集(CUB-200)上趋向于更连续，而在更简单的数据集(Oxford-IIIT Pet)上趋向于更稀疏。这是因为在特征提取中，两个相邻的张量往往具有耦合功能，如果选择其中任何一个，则应该始终都进行重新训练。通过进一步比较图(a)和(b)，更高的加速比ρ可以帮助在选择中消除这种限制，弹性变得更加显著。

内存开销

由于Jetson TX2上的CPU和GPU共享相同的内存，作者分别统计CPU和GPU内存使用情况。如图22所示，在Jetson TX2上，ElasticTrainer的内存消耗与所有其他方案相当，并且由于TensorFlow的限制，其CPU内存使用量略高于全训练，这是因为TensorFlow在每次张量选择后都会重新生成计算图，但不会释放之前计算图使用的内存。这种内存泄漏在只有cpu的Raspberry Pi设备上变得更加明显，其中Elastic-Trainer比完整训练多消耗10%的内存。然而，这样的内存消耗仍然保持在嵌入式设备的内存容量之内

能源开销

与全训练相比，ElasticTrainer将能量消耗减少了3倍